

estadisticas_user.js

Pantalla del lado del alumno que muestra, para un mes elegido, estadísticas calculadas a partir de su historial de entrenamientos: cuantas veces entreno, cuantos ejercicios hizo y como se reparten esos ejercicios entre grupos musculares. A diferencia de `mi_rutina_user.js` (que muestra la rutina ASIGNADA, es decir lo que el alumno debería hacer), esta pantalla muestra lo que el alumno REALMENTE hizo, reconstruido a partir de los registros guardados en `workoutLogs.js` cada vez que entrena.

1. El dato de origen: workoutLogs.js

Toda la pantalla se apoya en un archivo nuevo, `src/data/workoutLogs.js`, que simula la tabla de la base de datos donde quedaría el historial de entrenamientos de cada alumno. Cada registro tiene esta forma:

```
{
  id: 1,
  userId: 2,
  date: '2026-08-05',
  exercises: [
    { exerciseId: 2, muscleGroup: 'Abdomen' },
    { exerciseId: 1, muscleGroup: 'Abdomen' },
    ...
  ],
}
```

Esto representa dos tablas relacionadas, como quedarían en una base de datos real:

`registros_entrenamiento` (`id`, `id_usuario`, `fecha`) y `registro_ejercicios` (`id_registro`, `id_ejercicio`, `grupo_muscular`), unidas por `id_registro`. En el archivo JS la segunda tabla aparece anidada dentro de la primera (el array `exercises`), como quedaría el resultado de un JOIN entre ambas.

Un detalle importante: cada ejercicio del registro guarda una COPIA del grupo muscular (`muscleGroup`), en vez de salir a buscarlo a `exercises.js` en el momento de mostrar la pantalla. Es una decisión deliberada: si en el futuro un entrenador edita la rutina o se le cambia el grupo muscular a un ejercicio, el historial de meses anteriores no debe cambiar. La estadística de agosto tiene que seguir reflejando lo que realmente paso en agosto, aunque la rutina actual ya sea otra. Es el mismo motivo por el que una factura real guarda el precio del producto en el momento de la venta, en vez de recalcularlo con el precio de hoy.

El archivo expone una sola función, `getWorkoutLogsByUser(userId)`, que filtra el array completo por usuario (equivale a un `SELECT ... WHERE id_usuario = ?`). Los datos de este archivo son de ejemplo: están generados con un script que reparte entrenamientos de agosto y septiembre 2026 para cada alumno, usando los días reales de su rutina, para poder probar la pantalla antes de tener el QR implementado.

2. Estado: que mes se esta mirando

```
const today = new Date();
const [selected, setSelected] = useState({
  year: today.getFullYear(),
  month: today.getMonth(),
});
```

A diferencia del resto de las pantallas del alumno (que no tienen estado propio, solo leen datos), esta necesita recordar que mes está eligiendo el usuario mientras navega con las flechas. Arranca siempre en

el mes actual. Cada vez que se toca una flecha, `changeMonth(delta)` crea una fecha nueva sumando o restando un mes y actualiza `selected`, lo que dispara un re-render con los datos del mes correspondiente.

```
function changeMonth(delta) {
  const date = new Date(selected.year, selected.month + delta, 1);
  setSelected({ year: date.getFullYear(), month: date.getMonth() });
}
```

Pasar siempre el día 1 al constructor de `Date` evita un bug clásico: si hoy fuera 31 de un mes largo y se resta un mes, JavaScript 'desborda' hacia el mes siguiente (ej: 31 de abril no existe y se convierte en 1 de mayo). Al fijar el día en 1, el cálculo del mes siempre da el resultado esperado.

3. Filtrado del historial por usuario y por mes

```
const userLogs = getWorkoutLogsByUser(user.id);
const monthLogs = userLogs.filter((log) => log.date.startsWith(monthPrefix));
```

Primero se traen TODOS los registros del usuario logueado (`useAuth()`, igual que en `mi_rutina_user.js`) y después se filtran los que pertenecen al mes seleccionado. `monthPrefix` es el string 'YYYY-MM' del mes elegido (por ejemplo '2026-09'), y como las fechas de los registros tienen el formato 'YYYY-MM-DD', con `startsWith` alcanza para filtrar por mes sin tener que parsear cada fecha.

Las flechas del selector de mes no tienen ninguna restricción: se puede navegar libremente hacia atrás y hacia adelante. Si el mes elegido no tiene registros (por ejemplo, un mes futuro o uno anterior a que el alumno empezara a entrenar), la pantalla simplemente muestra el mensaje de 'mes sin registros' explicado en la sección 7, en vez de bloquear la navegación.

4. Resumen del mes: números simples

La primera caja (Resumen del mes) muestra tres números grandes, calculados con operaciones básicas sobre `monthLogs`:

```
monthLogs.length // entrenamientos
monthLogs.reduce((total, log) => total + log.exercises.length, 0) // ejercicios hechos
muscleGroups.length // grupos musculares distintos
```

La cantidad de entrenamientos es simplemente cuantos registros hay ese mes. Los ejercicios hechos se obtienen sumando, registro por registro, cuantos ejercicios tiene cada uno (`reduce`). Los grupos musculares distintos salen de la función `countByMuscleGroup`, explicada en la sección 6.

5. Grafico 'Entrenamientos por semana'

Este gráfico muestra, semana a semana dentro del mes, cuantos días entreno el alumno. No se compara contra ninguna meta: se descarto a propósito comparar contra la cantidad de días de la rutina asignada, porque esa rutina puede cambiar mes a mes (el entrenador la edita, el alumno cambia de plan), y una meta basada en la rutina ACTUAL no tendría sentido aplicada a meses pasados. Mantenerlo simple (solo mostrar lo que efectivamente paso) evita esa inconsistencia.

La función `getWeeksOfMonth()` arma las semanas del mes (de lunes a domingo) recorriendo día por día:

```
function getWeeksOfMonth(year, month, monthLogs) {
  const daysInMonth = new Date(year, month + 1, 0).getDate();
  const trainedDates = new Set(monthLogs.map((log) => log.date));
```

```

const weeks = [];
let week = null;

for (let day = 1; day <= daysInMonth; day++) {
  if (!week || new Date(year, month, day).getDay() === 1) {
    week = { from: day, to: day, trained: 0 };
    weeks.push(week);
  }
  week.to = day;
  if (trainedDates.has(toDateKey(year, month, day))) week.trained++;
}

return weeks;
}

```

El truco esta en `new Date(year, month, day).getDay()`: JavaScript devuelve 0 para domingo y 1 para lunes. Cada vez que el dia cae lunes (o es el primer dia del recorrido), se cierra la semana anterior y arranca una nueva. `trainedDates` es un Set con todas las fechas en que el usuario entreno ese mes, para poder preguntar 'entreno este dia' en tiempo constante en vez de recorrer el array de logs una vez por cada dia del mes.

`new Date(year, month + 1, 0).getDate()` es el truco estandar de JavaScript para saber cuantos dias tiene un mes: el 'dia 0' del mes siguiente es, en la practica, el ultimo dia del mes actual.

En el render, cada semana es una columna con una sola barra amarilla, armada con un View comun (sin ninguna libreria de graficos). Su altura es un porcentaje de `WEEK_CHART_HEIGHT` (120px), calculado sobre la semana con mas entrenamientos del mes, para que esa barra siempre llegue al tope del grafico:

```

const maxWeekValue = Math.max(1, ...weeks.map((w) => w.trained));
...
<View style={[styles.weekTrainedBar, { height: (week.trained / maxWeekValue) * WEEK_CHART_HEIGHT }]} />

```

Arriba de cada columna se muestra el valor como texto (la cantidad de dias entrenados esa semana). Al ser una unica serie de datos, no hace falta leyenda: el titulo de la seccion ya dice que representa el grafico.

6. Grafico 'Ejercicios por grupo muscular'

`countByMuscleGroup()` recorre todos los registros del mes y cuenta cuantas veces aparece cada grupo muscular entre los ejercicios hechos, usando un objeto como acumulador (equivalente a un `GROUP BY grupo_muscular` en SQL):

```

function countByMuscleGroup(monthLogs) {
  const counts = {};
  monthLogs.forEach((log) => {
    log.exercises.forEach((exercise) => {
      counts[exercise.muscleGroup] = (counts[exercise.muscleGroup] ?? 0) + 1;
    });
  });

  return Object.entries(counts)
    .map(([group, count]) => ({ group, count }))
    .sort((a, b) => b.count - a.count);
}

```

El resultado es un array ordenado de mayor a menor cantidad, para que el grupo mas trabajado aparezca siempre primero. En el render, cada grupo es una fila con su nombre a la izquierda, una barra horizontal cuyo ancho es un porcentaje ($\text{count} / \text{maxGroupCount}$) y el numero a la derecha. Igual que en el grafico semanal, no se usa ninguna libreria: el ancho de la barra es simplemente un View con un porcentaje en su estilo.

Se eligieron barras horizontales en vez de un grafico de torta a proposito: con hasta 10 grupos musculares posibles (definidos en `exercises.js`), una torta se vuelve dificil de leer, mientras que una lista de barras ordenadas permite ver de un vistazo cual es el grupo mas y menos trabajado, y compararlos facilmente entre si.

7. Mes sin registros

Si el mes elegido no tiene ningun registro (`monthLogs.length === 0`), la pantalla no dibuja graficos vacios: en su lugar muestra un unico mensaje ('No hay entrenamientos registrados en este mes'). Esto evita, por ejemplo, un grafico de barras en 0 o una division por cero al calcular porcentajes sobre un maximo que seria 0.

8. Integracion con el resto de la app

La pantalla se agrego al stack de navegacion del alumno en `app/index.tsx`, junto al resto de las pantallas de usuario comun, y se sumo una quinta pestania 'Estadisticas' en `UserBottomNav.js`, entre Reloj y Mi Cuenta, con el mismo patron de icono + texto que las demas pestanias (resaltado en amarillo cuando `activeScreen === 'stats'`). Ademàs, en `home_user.js` se agrego una caja nueva 'Ver mas estadisticas' con un boton 'Ver' que navega directo a esta pantalla, siguiendo el mismo estilo visual que las otras cajas del inicio (Estadisticas, Progreso diario, Tu rutina).

9. Estilos

Mismo lenguaje visual que el resto de la app: fondo general #101010, cajas de seccion ('sectionBox') con borde amarillo palido #FFE082 y esquinas redondeadas, igual que en `mi_rutina_user.js` y `home_user.js`. El amarillo principal (#FFC107) se usa tanto para los numeros destacados del resumen como para las barras de ambos graficos, manteniendo coherencia entre todos los elementos que representan 'lo que el usuario hizo'.